Lecture 07: Python Basics (Chapter 4 & 5)

교재 Ch. 5.1, 5.4, 5.5

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 12일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

5.1 For Loop

반복문

For Loop: 컨테이너 순회

`for` 루프는 문자열, 리스트, 숫자 범위와 같은 **컨테이너(Container)** 내부의 요소를 하나씩 꺼내며 반복합니다.

- **문자열 순회:** 한 글자씩(Char) 꺼내기
- **리스트 순회:** 명부에서 이름 하나씩 꺼내기

문자열 글자 수 세기 예시

```
str_var = "Python"
for c in str_var:
    print(c, end="*")
# 출력 결과: P*y*t*h*o*n*
```

For Loop: 반복의 정석 (Repetition)

특정 횟수만큼 작업을 반복하고 싶을 때, `for` 루프는 가장 직관적인 해결책입니다. 보통 `range()` 함수와 결합하여 사용합니다.

5번 반복하여 인사하기

```
for i in range(5):    # 0, 1, 2, 3, 4를 순차적으로 생성
    print("Hello Python!")
```

- **반복의 원리:** 파이썬은 "5번 해라"라고 말하는 대신, "숫자 5개가 든 주머니에서 하나씩 꺼낼 때마다 한 번씩 실행해라"라고 이해합니다.
- **변수 `i`의 역할:** 루프가 돌 때마다 현재 몇 번째 반복인지 알려주는 '카운터' 역할을 합니다.

range() 함수: 숫자 사슬 생성기

정해진 횟수만큼 숫자를 생성하여 `for` 문에 전달합니다.

range() 함수의 세 가지 얼굴

형식	설명	예시 (결과)
`range(end)`	0부터 end-1까지 생성	`range(3)` → 0, 1, 2
`range(s, e)`	s부터 e-1까지 생성	`range(2, 5)` → 2, 3, 4
`range(s, e, t)`	t 간격으로 생성	`range(1, 6, 2)` → 1, 3, 5

주의사항 (Exclusive End)

마지막 숫자인 end는 **포함되지 않습니다!** 1부터 10까지 출력하고 싶다면 `range(1, 11)`을 써야 합니다.

for vs while: 효율성 비교

동일한 작업을 할 때 `for` 문은 `while` 문보다 훨씬 간결(Compact)합니다.

While 방식 (초기화+업데이트)

```
i = 0
while i < 50:
  print(i)
  i += 5
```

For 방식 (한 줄 정리)

```
for i in range(0, 50, 5):
  print(i)
```

"코드가 짧을수록 버그가 생길 확률도 줄어듭니다!"

예제 1: 공백 사냥꾼 (Counting Spaces)

Mission: 입력받은 문장에 포함된 공백(' ')의 개수를 세어보세요.

1. `input()`으로 문장을 하나 입력받습니다.
2. `for` 문을 사용하여 문장의 한 글자씩 검사합니다.
3. 글자가 공백(`" "`)일 때마다 `count` 변수를 1씩 증가시킵니다.
4. 최종 공백 개수를 출력하세요.

실행 예시

`Input: "Hi everyone"` → `Output: 1`

5.4 Break and Continue

루프의 흐름을 지배하는 제어문

Break: "지금 당장 멈춰!"

`break` 문은 반복문(for, while) 내부에서 특정 조건이 발생했을 때, **루프 전체를 즉시 종료**하고 빠져나옵니다.

실제 사례: 특정 문자 찾기 (Search)

```
user_string = "This is a string."
for i in range(len(user_string)):
    if user_string[i] == 'a':
        print("Found at index:", i)
        break # 찾았으므로 나머지는 더 볼 필요 없음!
```

- **효율성:** 100만 명의 명단에서 3번째 사람을 찾았다면, 남은 99만 9,997번의 반복을 건너뛰어 성능을 아낍니다.

무한 루프와 Break의 조합

조건이 항상 True인 루프(Infinite Loop)를 만든 뒤, 내부에서 특정 조건이 맞을 때만 break로 탈출시키는 방식은 실전에서 매우 자주 쓰입니다.

While True 구조

```
counter = 1
while True:
  if counter >= 10:
    break
  print(counter)
  counter += 1
```

보안 팁

보안 모니터링 시스템은 항상 실행 상태(True)여야 합니다. 단, 시스템 종료 신호가 들어올 때만 break로 안전하게 닫아야 하죠.

Continue: "이번 판만 쉬고 다음으로!"

`continue` 문은 루프를 아예 끝내는 것이 아니라, **현재 차례부터 남은 코드만 건너뛰고** 다음 반복(Iteration)을 시작합니다.

홀수만 건너뛰기 예시

```
for i in range(5):
    if i % 2 != 0:
        continue # 홀수면 아래 print를 무시하고 다음 숫자로!
    print(i)
# 출력 결과: 0, 2, 4
```

예시 1: 공백 탐지기 (Space Hunter)

Mission: 문장에 공백이 포함되어 있는지 확인하세요.

1. 문자열을 입력받습니다.
2. `for` 문으로 돌면서 공백(`' '`)을 발견하면 "Found"를 출력하고 **즉시 루프를 탈출**하세요.
3. 공백이 전혀 없으면 "Not found"를 출력하세요.

5.5 Loop Else

루프가 '완주'했을 때만 실행되는 블록

Loop Else: 루프의 무사 완주 기념

`else` 문은 `for`나 `while` 루프가 **break 문에 의해 중단되지 않고** 끝까지 실행되었을 때만 실행됩니다.

- **작동 조건:** 루프의 조건식이 `False`가 되어 자연스럽게 종료될 때.
- **미작동 조건:** 루프 내부에서 `break`를 만나 강제로 탈출할 때.

주의: 들여쓰기(Indentation)

이 `else`는 `if`의 짝꿍이 아닙니다! `for`/`while` **문과 수직으로 정렬**되어야 합니다.

비교: 깃발(Flag) 변수 vs Loop Else

리스트에서 특정 값을 찾지 못했을 때의 처리를 비교해 봅시다.

전통적인 방식 (Flag)

```
found = False
for i in numbers:
  if i == 10:
    print("Found!")
    found = True; break
if not found:
  print("Not in list.")
```

파이썬 방식 (Loop Else)

```
for i in numbers:
  if i == 10:
    print("Found!")
    break
else:   # 완주 시에만!
  print("Not in list.")
```

"코드가 더 짧아지고 의도가 명확해집니다."

while-else 예시: 2의 거듭제곱 판별

입력값이 2의 거듭제곱인지 확인하기

```
i = n
while i > 1:
  if n % 2 == 0: i /= 2
  else: break # 홀수가 섞여있으면 중단
else:
  print(f"{n}은 2의 거듭제곱입니다!")
```

- 숫자가 계속 2로 나누어져서 결국 1이 되면(완주), `else` 문이 실행됩니다.
- 중간에 나누어떨어지지 않아 `break`가 걸리면, `else`는 무시됩니다.

Any Questions?